

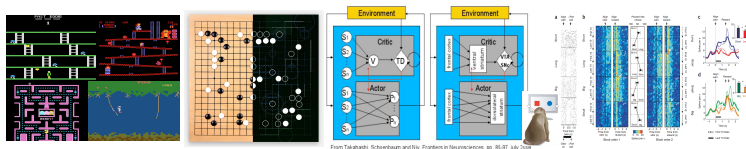
Tabular Reinforcement Learning

Olivier Sigaud

Sorbonne Université
<http://people.isir.upmc.fr/sigaud>



Why this class?



- ▶ A lot of buzz about deep reinforcement learning as an engineering tool
- ▶ The reinforcement learning framework is also relevant in computational neuroscience and psychology



Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. (2015) Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.

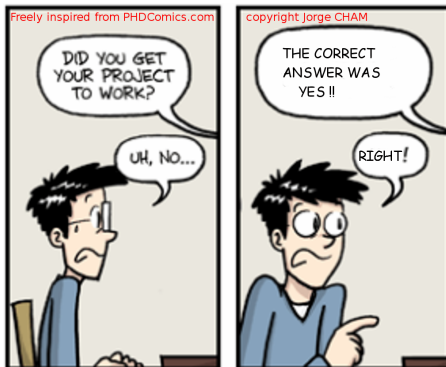


Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., et al. (2017) Mastering the game of go without human knowledge. *Nature*, 550(7676):354–359



Roesch, M. R., Calu, D. J., & Schoenbaum, G. (2007) Dopamine neurons encode the better option in rats deciding between differently delayed or sized rewards. *Nature Neuroscience*, 10(12):1615–1624

Supervised learning



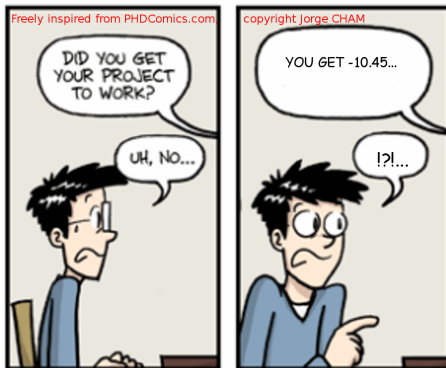
- ▶ The supervisor indicates to the agent **the expected answer**
- ▶ The agent **corrects a model** based on the answer
- ▶ Typical mechanism: gradient backpropagation, RLS
- ▶ Applications: classification, regression, function approximation...

Cost-Sensitive Learning



- ▶ The environment provides **the value of action** (reward, penalty)
- ▶ Application: behaviour optimization

Reinforcement learning



- ▶ In RL, the value signal is given as a scalar
- ▶ How good is -10.45?
- ▶ Necessity of exploration

The exploration/exploitation trade-off



- ▶ Exploring can be (very) harmful
- ▶ Shall I exploit what I know or look for a better policy?
- ▶ Am I optimal? Shall I keep exploring or stop?
- ▶ Decrease the rate of exploration along time
- ▶ ϵ -greedy: take the best action most of the time, and a random action from time to time

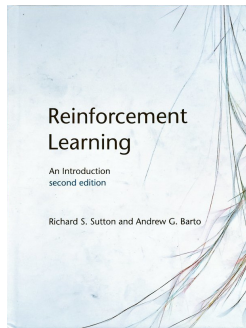
Sequentiality: Bandits problems vs Sequential problems



s_0	0.48	0.53	0.59	0.66	0.73
s_1	0.43	0.48	0.53		0.81
	0.39		0.48		0.9
	0.35	0.39	0.43		1

- ▶ A multi-armed bandit problem is a one step/state RL problem where each arm is an action (and reward is stochastic for more fun)
- ▶ RL problem are sequential decision making problems
- ▶ The state at step $t + 1$ depends on the action at state t
- ▶ This makes exploration/optimization much more difficult

Introductory book: the bible



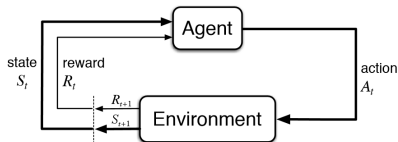
- ▶ [Sutton and Barto, 2018]: the ultimate introduction to the field, in the discrete case
- ▶ Available online:
<https://drive.google.com/file/d/1xeUDVGWGUUv1-ccUMAZHJLej2C7aAFWY/view>
- ▶ First version in 1998



Sutton, R. S. & Barto, A. G. (2018) *Reinforcement Learning: An Introduction*. MIT Press.

Markov Decision Processes

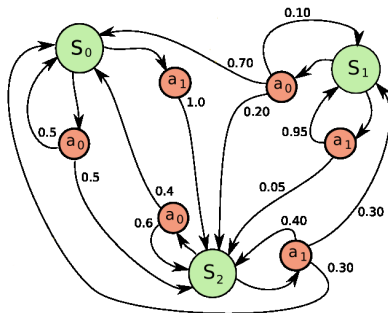
Markov Decision Processes



- ▶ S : state space
- ▶ A : action space
- ▶ $T : S \times A \rightarrow \Pi(S)$: transition function
- ▶ $r : S \times A \rightarrow \mathbb{R}$: reward function

- ▶ An MDP describes a problem, not a solution to that problem

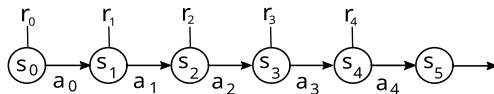
Stochastic transition function



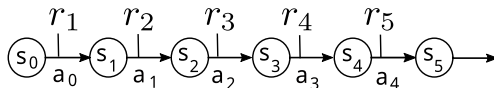
- Deterministic problem = special case of stochastic
- $T(s^t, a^t, s^{t+1}) = p(s'|s, a)$

Rewards: over states or action?

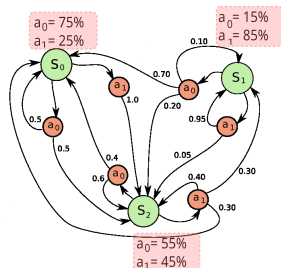
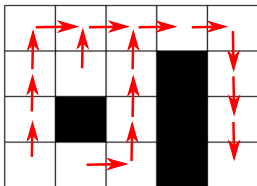
► Reward over states



► Reward over actions in states



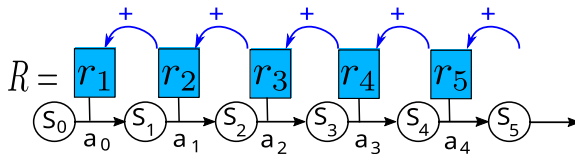
Deterministic versus stochastic policy



- ▶ The agent is driven by a **policy**
- ▶ Goal: find a policy $\pi : S \rightarrow A$ maximizing an aggregation of rewards (the **return** R) on the long run
- ▶ The policy itself can be deterministic or stochastic
- ▶ Important theorem: for any MDP, there exists a deterministic policy that is optimal

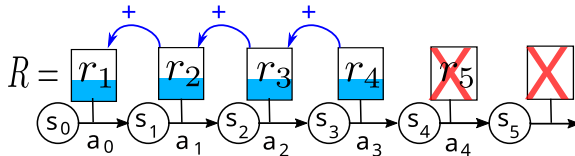
Agregation criterion: mere sum

- The computation of value functions assumes the choice of an aggregation criterion (discounted, average, etc.)



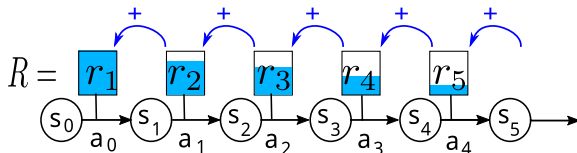
- The sum over a infinite horizon may be infinite, thus hard to compare
- Mere sum (finite horizon N): $V^\pi(S_0) = r_1 + r_2 + \dots + r_N$

Agregation criterion: average over a window



- Average criterion on a window: $V^\pi(s_0) = \frac{r_1 + r_2 + r_3 + r_4}{4} \dots$

Agregation criterion: discounted

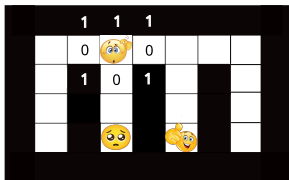


- ▶ Discounted criterion: $V^\pi(s_{t_0}) = \sum_{t=t_0}^{\infty} \gamma^t r(s_t, \pi(s_t))$
- ▶ $\gamma \in [0, 1]$: discount factor
 - ▶ if $\gamma = 0$, sensitive only to immediate reward
 - ▶ if $\gamma = 1$, future rewards are as important as immediate rewards
- ▶ The discounted case is the most used
- ▶ The discount factor can be a parameter of the agent or the environment

Markov Property

- ▶ A deterministic MDP defines s_{t+1} and r_{t+1} as $f(s_t, a_t)$
- ▶ **Markov property** : $p(s_{t+1}|s_t, a_t) = p(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0)$
- ▶ In an MDP, a memory of the past does not provide any useful advantage
- ▶ **Reactive agents** $a_{t+1} = f(s_t)$, without internal states nor memory, can be optimal

Markov property: Limitations



- ▶ Markov property is not verified if:
 - ▶ the observation does not contain all useful information to take decisions (POMDPs)
 - ▶ or if the next state depends on decisions of several agents (Dec-MDPs, Dec-POMDPs, Markov games)
 - ▶ or if transitions depend on time (Non-stationary problems)

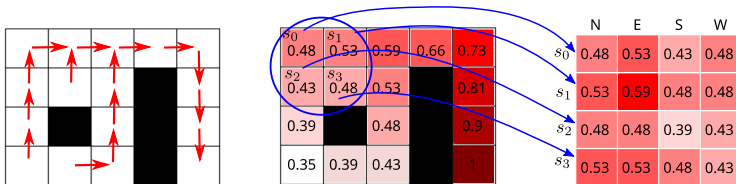
Dynamic Programming

Dynamic Programming

- ▶ Once we have defined an MDP
- ▶ Dynamic programming methods can find the optimal policy
- ▶ Assuming they know everything about the MDP
- ▶ Reinforcement Learning applies when the transition and reward functions are unknown
- ▶ To define dynamic programming methods, we need value functions

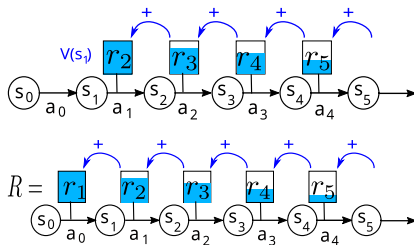
Policy-dependent value and action-value functions

Policy-dependent value and action value functions



- For each state, the **value function** $V^\pi : S \rightarrow \mathbb{R}$ records the return on the long run **following policy** π .
- $V^\pi(s) = \mathbb{E}_\pi[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} | s_t = s]$
- It is a **vector** with one entry per state
- For each (state, action) pair, the **action value function** $Q^\pi : S \times A \rightarrow \mathbb{R}$ records the return on the long run for doing this action in this state **and then following policy** π .
- $Q^\pi(s, a) = \mathbb{E}_\pi[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} | s_t = s, a_t = a]$
- It is a **matrix** with one entry per state and per action
- In the remainder, we focus on V , trivial to transpose to Q

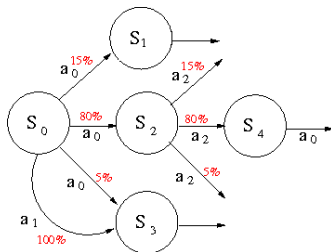
Bellman equation over a Markov chain: recursion



Given the discounted reward aggregation criterion:

- ▶ $V(s_0) = r_1 + \gamma r_2 + \gamma^2 r_3 + \gamma^3 r_4 + \dots$
- ▶ $V(s_0) = r_1 + \gamma(r_2 + \gamma r_3 + \gamma^2 r_4 + \dots)$
- ▶ $V(s_0) = r_1 + \gamma V(s_1)$
- ▶ More generally $V(s_t) = r_{t+1} + \gamma V(s_{t+1})$

Bellman equation over a stochastic tree



- ▶ Generalisation of $V(s_t) = r_{t+1} + \gamma V(s_{t+1})$ over all possible trajectories
- ▶ The **expectation** of a random variable is the sum of the realizations weighted by their probabilities
- ▶ Deterministic π : $V^\pi(s) = r(s, \pi(s)) + \gamma \sum_{s'} p(s'|s, \pi(s)) V^\pi(s')$
 - ▶ Realizations are the next states
- ▶ Stochastic π : $V^\pi(s) = \sum_a \pi(a|s) [r(s, a) + \gamma \sum_{s'} p(s'|s, a) V^\pi(s')]$
 - ▶ Realizations are the chosen actions and the next states

Relations between V and Q

$$V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s, a)$$

Can be seen as a sort of expectation over Q-values

$$Q^\pi(s, a) = \sum_{s'} p(s'|s, a) [r(s, a) + \gamma V(s')]$$

- By application of one step Bellman recursion



Puterman, M. L. (2014) *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons.

Optimal value and action-value functions, optimal policy

Optimal values and optimal policy

- ▶ The optimal value and action-value functions are noted V^* and Q^*
- ▶ They are defined as:



$$\forall s \in S, V^*(s) = \max_{\pi} V^{\pi}(s)$$



$$\forall (s, a) \in S \times A, Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a)$$

- ▶ The optimal policy π^* is the policy that achieves optimal value and optimal action-value functions:



$$\pi = \pi^* \iff V^{\pi} = V^* \iff Q^{\pi} = Q^*$$

- ▶ Now the question is: how do we find V^* and Q^* ?

Recursive operators and convergence

- ▶ If we define an operator T such that $X_{n+1} \leftarrow TX_n$
- ▶ If T is **contractive**, then through repeated application of T , X_n will converge to some fixed point
- ▶ For instance, if T divides by 2, X_n converges to 0

The Bellman optimality operator (Value Iteration)

- ▶ We call **Bellman optimality operator** (noted T^*) the application

$$\forall s \in S, V_{n+1}(s) \leftarrow \max_{a \in A} \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V_n(s') \right]$$

- ▶ If $\gamma < 1$, T^* is contractive
- ▶ By iterating, computes the value of the current policy
- ▶ The optimal value function is the fixed-point of T^* : $V^* = T^*V^*$
- ▶ Value iteration: start from any V_0 , then $V_{n+1} \leftarrow T^*V_n$

Value Iteration: the algorithm

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
 Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

```
|  $\Delta \leftarrow 0$ 
| Loop for each  $s \in \mathcal{S}$ :
|    $v \leftarrow V(s)$ 
|    $V(s) \leftarrow \max_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$ 
|    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
until  $\Delta < \theta$ 
```

Output a deterministic policy, $\pi \approx \pi_*$, such that

$$\pi(s) = \arg \max_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$$

- ▶ Taken from Sutton & Barto, 2018, p. 83
- ▶ Reminder: $\sum_{s',r} p(s', r | s, a) [r + \gamma V(s')] = r + \gamma \sum_{s'} p(s' | s, a) V(s')$
- ▶ (because r is deterministic)

Value Iteration in practice

0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0		0.0
0.0		0.0		0.0
0.0	0.0	0.0		1

$$\forall s \in S, V_{i+1}(s) \leftarrow \max_{a \in A} \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V_i(s') \right]$$

Value Iteration in practice

0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0		0.0
0.0		0.0		0.9
0.0	0.0	0.0		1

$$\forall s \in S, V_{i+1}(s) \leftarrow \max_{a \in A} \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V_i(s') \right]$$

Value Iteration in practice

0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0		0.81
0.0		0.0		0.9
0.0	0.0	0.0		1

$$\forall s \in S, V_{i+1}(s) \leftarrow \max_{a \in A} \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V_i(s') \right]$$

Value Iteration in practice

0.0	0.0	0.0	0.0	0.73
0.0	0.0	0.0		0.81
0.0		0.0		0.9
0.0	0.0	0.0		1

$$\forall s \in S, V_{i+1}(s) \leftarrow \max_{a \in A} \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V_i(s') \right]$$

Value Iteration in practice

0.0	0.0	0.0	0.66	0.73
0.0	0.0	0.0		0.81
0.0		0.0		0.9
0.0	0.0	0.0		1

$$\forall s \in S, V_{i+1}(s) \leftarrow \max_{a \in A} \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V_i(s') \right]$$

Value Iteration in practice

0.0	0.0	0.59	0.66	0.73
0.0	0.0	0.0		0.81
0.0		0.0		0.9
0.0	0.0	0.0		1

$$\forall s \in S, V_{i+1}(s) \leftarrow \max_{a \in A} \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V_i(s') \right]$$

Value Iteration in practice

0.0	0.53	0.59	0.66	0.73
0.0	0.0	0.53		0.81
0.0		0.0		0.9
0.0	0.0	0.0		1

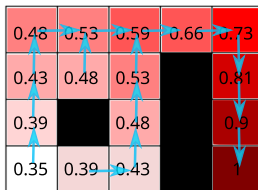
$$\forall s \in S, V_{i+1}(s) \leftarrow \max_{a \in A} \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V_i(s') \right]$$

Value Iteration in practice

0.48	0.53	0.59	0.66	0.73
0.43	0.48	0.53		0.81
0.39		0.48		0.9
0.35	0.39	0.43		1

$$\forall s \in S, V_{i+1}(s) \leftarrow \max_{a \in A} \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V_i(s') \right]$$

Value Iteration in practice



- We stop once changes are below a threshold
- Given a V function, determine a deterministic policy by

$$\pi(s) = \arg \max_{a \in A} \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V_i(s') \right]$$
- Given a Q function, we do not need to represent the policy: just choose

$$\pi(s) = \arg \max_{a \in A} Q(s, a)$$

The Bellman operator (Policy Iteration)

- ▶ We call **Bellman operator** (noted T^π) the application

$$\forall s \in S, V_{n+1}^\pi(s) \leftarrow r(s, \pi(s)) + \gamma \sum_{s'} p(s'|s, \pi(s)) V_n^\pi(s')$$

- ▶ If $\gamma < 1$, T is contractive
- ▶ Policy Iteration:
 - ▶ Start from any policy π_0
 - ▶ Policy evaluation:
 $V_{n+1}^\pi \leftarrow T^\pi V_n^\pi$
 - ▶ Policy improvement:
 $\forall s \in S, \pi_{n+1}(s) \leftarrow \arg \max_{a \in A} [r(s, a) + \gamma \sum_{s'} p(s'|s, a) V_n^\pi(s')]$
- ▶ Converges to optimal value and policy
- ▶ Does not make much sense when using Q (the critic and the policy are the same)

Policy Iteration: the algorithm

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

policy-stable $\leftarrow$ true

For each $s \in \mathcal{S}$:

old-action $\leftarrow \pi(s)$

$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ false

If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

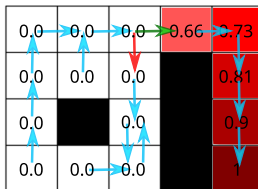
- Taken from Sutton & Barto, 2018, p. 80
- Note: $\sum_{s',r} p(s',r|s,a) [r + \gamma V(s')] = r + \gamma \sum_{s'} p(s'|s,a) V(s')$
- (because r is deterministic)

Policy Iteration in practice

0.0	0.0	0.0	0.66	0.73
0.0	0.0	0.0		0.81
0.0		0.0		0.9
0.0	0.0	0.0		1

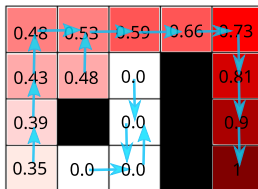
$$\forall s \in S, V_i(s) \leftarrow \text{evaluate}(\pi_i(s))$$

Policy Iteration in practice



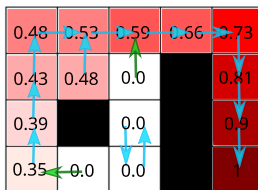
$$\forall s \in S, \pi_{i+1}(s) \leftarrow \text{improve}(\pi_i(s), V_i(s))$$

Policy Iteration in practice



$$\forall s \in S, V_i(s) \leftarrow \text{evaluate}(\pi_i(s))$$

Policy Iteration in practice



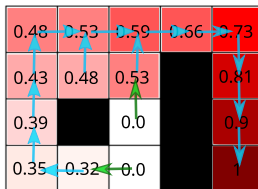
$$\forall s \in S, \pi_{i+1}(s) \leftarrow \text{improve}(\pi_i(s), V_i(s))$$

Policy Iteration in practice

0.48	0.53	0.59	0.66	0.73
0.43	0.48	0.53		0.81
0.39		0.0		0.9
0.35	0.32	0.0		1

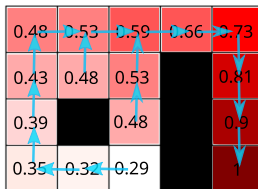
$$\forall s \in S, V_i(s) \leftarrow \text{evaluate}(\pi_i(s))$$

Policy Iteration in practice



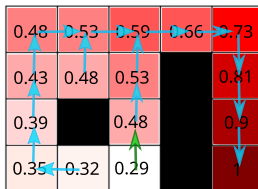
$$\forall s \in S, \pi_{i+1}(s) \leftarrow \text{improve}(\pi_i(s), V_i(s))$$

Policy Iteration in practice



$$\forall s \in S, V_i(s) \leftarrow \text{evaluate}(\pi_i(s))$$

Policy Iteration in practice



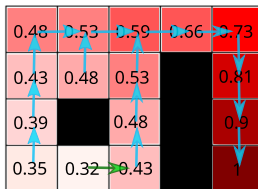
$$\forall s \in S, \pi_{i+1}(s) \leftarrow \text{improve}(\pi_i(s), V_i(s))$$

Policy Iteration in practice

0.48	0.53	0.59	0.66	0.73
0.43	0.48	0.53		0.81
0.39		0.48		0.9
0.35	0.32	0.43		1

$$\forall s \in S, V_i(s) \leftarrow \text{evaluate}(\pi_i(s))$$

Policy Iteration in practice



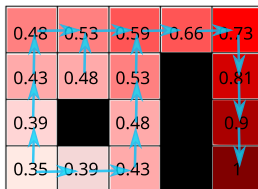
$$\forall s \in S, \pi_{i+1}(s) \leftarrow \text{improve}(\pi_i(s), V_i(s))$$

Policy Iteration in practice

0.48	0.53	0.59	0.66	0.73
0.43	0.48	0.53		0.81
0.39		0.48		0.9
0.35	0.39	0.43		1

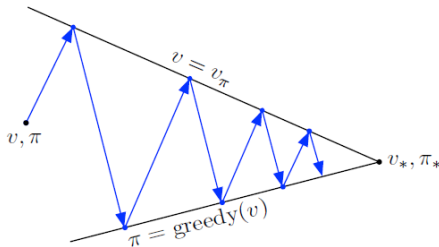
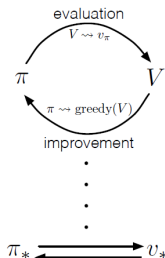
$$\forall s \in S, V_i(s) \leftarrow \text{evaluate}(\pi_i(s))$$

Policy Iteration in practice



Here we have managed a policy and a value representations at all steps

Generalized Policy Iteration



- Policy iteration evaluates each intermediate policy up to convergence.
This is slow.
- Instead, evaluate the policy for N iterations, or even not for all states.
- **Asynchronous dynamic programming:** decoupling policy evaluation and improvement
- Taken from Sutton & Barto, 2018

Temporal difference mechanisms

Reinforcement learning

- ▶ In Dynamic Programming (planning), T and r are given
- ▶ Reinforcement learning goal: build π^* without knowing T and r
- ▶ **Model-free approach**: build π^* without estimating T nor r
- ▶ **Actor-critic approach**: special case of model-free
- ▶ **Model-based approach**: build a model of T and r and use it to improve the policy

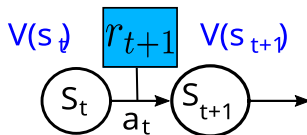
Incremental estimation

- ▶ Estimating the average immediate (stochastic) reward in a state s
- ▶ $E_k(s) = (r_1 + r_2 + \dots + r_k)/k$
- ▶ $E_{k+1}(s) = (r_1 + r_2 + \dots + r_k + r_{k+1})/(k+1)$
- ▶ Thus $E_{k+1}(s) = k/(k+1)E_k(s) + r_{k+1}/(k+1)$
- ▶ Or $E_{k+1}(s) = (k+1)/(k+1)E_k(s) - E_k(s)/(k+1) + r_{k+1}/(k+1)$
- ▶ Or $E_{k+1}(s) = E_k(s) + 1/(k+1)[r_{k+1} - E_k(s)]$
- ▶ Still needs to store k
- ▶ Can be approximated as $E_{k+1}(s) \leftarrow E_k(s) + \alpha[r_{k+1} - E_k(s)]$
- ▶ And rewritten

$$E_{k+1}(s) \leftarrow (1 - \alpha).E_k(s) + \alpha.r_{k+1} \quad (1)$$

- ▶ Converges to the true average (slower or faster depending on α) without storing anything
- ▶ Equation (1) is everywhere in reinforcement learning

Temporal Difference error



- ▶ The goal of TD methods is to estimate the value function $V(s)$
- ▶ If estimations $V(s_t)$ and $V(s_{t+1})$ were exact, we would get $V(s_t) = r_{t+1} + \gamma V(s_{t+1})$
- ▶ The approximation error is

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t) \quad (2)$$

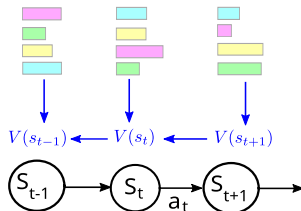
- ▶ δ_t measures the error between $V(s_t)$ and the value it should have given $r_{t+1} + \gamma V(s_{t+1})$
- ▶ If $\delta_t > 0$, $V(s_t)$ is under-evaluated, otherwise it is over-evaluated
- ▶ $V_{k+1}(s_t) \leftarrow V_k(s_t) + \alpha \delta_t$ should decrease the error (value propagation)

Temporal Difference update rule

$$E_{k+1}(s) = (1 - \alpha).E_k(s) + \alpha.r_{k+1} \quad (1)$$

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t) \quad (2)$$

$$V_{k+1}(s_t) \leftarrow (1 - \alpha).V_k(s_t) + \alpha.[r_{t+1} + \gamma V_k(s_{t+1})] \quad (3)$$



- By expanding δ in (2), we get

$$V_{k+1}(s_t) \leftarrow V_k(s_t) + \alpha[r_{t+1} + \gamma V_k(s_{t+1}) - V_k(s_t)]$$

- Or:

$$V_{k+1}(s_t) \leftarrow (1 - \alpha).V_k(s_t) + \alpha.[r_{t+1} + \gamma V_k(s_{t+1})] \quad (3)$$

- Combines two estimation processes:

- incremental estimation (1)
- value propagation from $V(s_{t+1})$ to $V(s_t)$ (2)

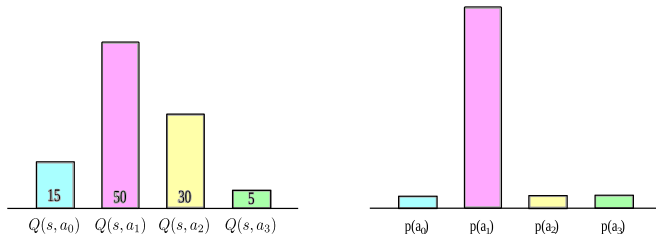
The Policy evaluation algorithm: TD(0)

- ▶ An agent performs a sequence
 $s_0, a_0, r_1, \dots, s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, r_{t+2}, \dots$
- ▶ Performs local Temporal Difference updates from s_t, s_{t+1} and r_{t+1}
- ▶ Proved in 1994 provided ϵ -greedy exploration
- ▶ Note: updates can be performed in any order



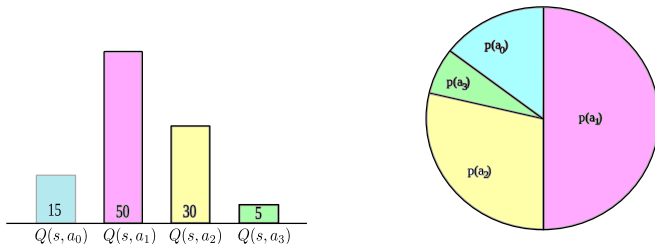
Dayan, P. & Sejnowski, T. (1994). TD(λ) converges with probability 1. *Machine Learning*, 14(3):295–301.

ϵ -greedy exploration



- ▶ Choose the best action with a high probability, other actions at random with low probability
- ▶ Same properties as random search
- ▶ Every state-action pair will be enough visited under an infinite horizon
- ▶ Useful for convergence proofs

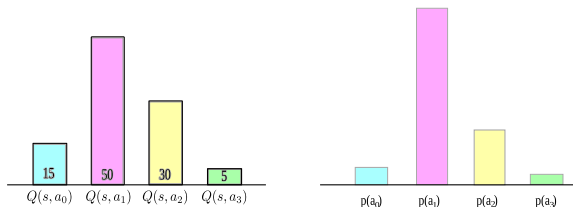
Roulette wheel



$$p(a_i) = \frac{Q(s, a_i)}{\sum_j Q(s, a_j)}$$

- The probability of choosing each action is proportional to its value

Softmax exploration



$$p(a_i) = \frac{e^{\frac{Q(s, a_i)}{\beta}}}{\sum_j e^{\frac{Q(s, a_j)}{\beta}}}$$

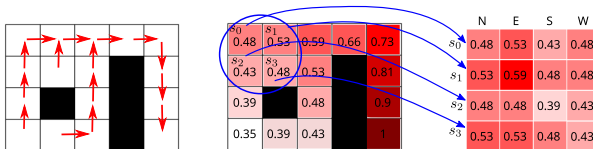
- ▶ The parameter β is called the temperature
- ▶ If $\beta \rightarrow \infty$, all actions have the same probability $\rightarrow$ random choice
- ▶ If $\beta \rightarrow 0$, increase contrast between values
- ▶ More used in computational neurosciences
- ▶ In machine learning RL, one often uses $\tau = \frac{1}{\beta}$, i.e. $p(a_i) = \frac{e^{\tau Q(s, a_i)}}{\sum_j e^{\tau Q(s, a_j)}}$
- ▶ Meta-learning: tune β dynamically (exploration/exploitation)

TD(0): limitation

- ▶ TD(0) evaluates $V(s)$
- ▶ One cannot infer $\pi(s)$ from $V(s)$ without knowing T : one must know which a leads to the best $V(s')$
- ▶ Three solutions:
 - ▶ Q-LEARNING, SARSA: Work with $Q(s, a)$ rather than $V(s)$.
 - ▶ ACTOR-CRITIC methods: Simultaneously learn V and update π
 - ▶ DYNA: Learn a model of T : model-based (or indirect) reinforcement learning

RL algorithms

Value function and Action Value function



- ▶ The **value function** $V^\pi : S \rightarrow \mathbb{R}$ records the aggregation of reward on the long run for each state (following policy π). It is a **vector** with one entry per state
- ▶ The **action value function** $Q^\pi : S \times A \rightarrow \mathbb{R}$ records the aggregation of reward on the long run for doing each action in each state (and then following policy π). It is a **matrix** with one entry per state and per action

SARSA

- ▶ Reminder (TD): $V(s_t) \leftarrow V(s_t) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$
- ▶ SARSA: For each observed $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$:
 $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$
- ▶ Policy: perform exploration (e.g. ϵ -greedy)
- ▶ One must know the action a_{t+1} , thus constrains exploration
- ▶ On-policy method: more complex convergence proof



Singh, S. P., Jaakkola, T., Littman, M. L., & Szepesvari, C. (2000). Convergence Results for Single-Step On-Policy Reinforcement Learning Algorithms. *Machine Learning*, 38(3):287–308.

SARSA: the algorithm

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Loop for each step of episode:

 Take action A , observe R, S'

 Choose A' from S' using policy derived from Q (e.g., ε -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

 until S is terminal

► Taken from Sutton & Barto, 2018

Q-LEARNING

- For each observed $(s_t, a_t, r_{t+1}, s_{t+1})$:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma \max_{a \in A} Q(s_{t+1}, a) - Q(s_t, a_t)]$$

- $\max_{a \in A} Q(s_{t+1}, a)$ instead of $Q(s_{t+1}, a_{t+1})$
- **Off-policy method**: no more need to know a_{t+1}
- Policy: perform exploration (e.g. ϵ -greedy)
- Convergence proven given infinite exploration



Watkins, C. J. C. H. (1989). *Learning with Delayed Rewards*. PhD thesis, Psychology Department, University of Cambridge, England.



Watkins, C. J. C. H. & Dayan, P. (1992) Q-learning. *Machine Learning*, 8:279–292

Q-LEARNING: the algorithm

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Take action A , observe R , S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

 until S is terminal

► Taken from Sutton & Barto, 2018

Q-LEARNING: details of the algorithm

Algorithm 1: Q-LEARNING

```
1: obs = env.reset()
2: total_reward = 0
3: done = False
4: while not done do
5:   action = agent.get_action(obs, deterministic)
6:   next_obs, reward, terminated, truncated, _ = env.step(action)
7:    $Q[\text{obs}, \text{action}] = Q[\text{obs}, \text{action}] + \alpha[\text{reward} + \gamma \max_{a'} Q[\text{next\_obs}, a'] - Q[\text{obs}, \text{action}]]$ 
8:   done = terminated or truncated
9:   obs = next_obs
10:  total_reward += reward
11: end while
12: return total_reward
```

- ▶ Note the gym(nasium) interface
- ▶ Why (1 - terminated)?

Episode termination

- ▶ Terminated: the episode finishes due to success or failure
- ▶ Truncated: the episode finishes because the time limit is reached
- ▶ $1 - \text{Terminated} = 0$ if Terminated is True, $= 1$ if Terminated is False
- ▶ $Q[\text{obs}, \text{action}] = Q[\text{obs}, \text{action}] + \alpha[\text{reward} + \gamma \max_{a'} Q[\text{next_obs}, a'] - Q[\text{obs}, \text{action}]]$
- ▶ Is equivalent to:

Algorithm 2: Bellman backup

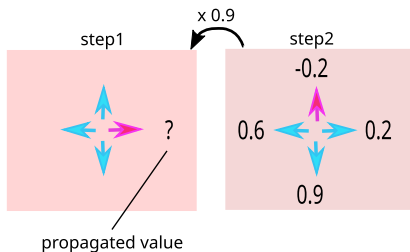
```
1: if Terminated then
2:    $Q[\text{obs}, \text{action}] = Q[\text{obs}, \text{action}] + \alpha[\text{reward} - Q[\text{obs}, \text{action}]]$ 
3: else
4:    $Q[\text{obs}, \text{action}] = Q[\text{obs}, \text{action}] + \alpha[\text{reward} + \gamma \max_{a'} Q[\text{next\_obs}, a'] - Q[\text{obs}, \text{action}]]$ 
5: end if
```

- ▶ If the episode is truncated, we should still bootstrap, as if the episode had continued



Pardo, F., Tavakoli, A., Levdiuk, V., and Kormushev, P. (2018) Time limits in reinforcement learning. In *International Conference on Machine Learning*, pages 4045–4054. PMLR

Difference between Q-LEARNING and SARSA



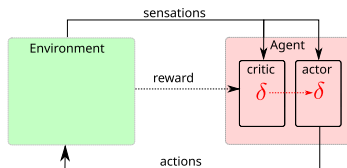
- Consider an agent taking the two pink actions
- With Q-LEARNING, the propagated value is $\gamma \arg\max_a Q(s_{t+1}, a)$, thus $0.9 \times 0.9 = 0.81$
- With SARSA, it is $\gamma Q(s_{t+1}, a_{t+1})$, thus $0.9 \times -0.2 = -0.18$

Q-LEARNING in practice

 0.0	 0.0	 0.0	 0.0	 0.0
 0.0	 0.0	 0.0		 0.0
 0.0		 0.0		 0.0
 0.0		 0.0		 0.0

- ▶ Build a $\text{states} \times \text{actions}$ table (*Q-Table*, eventually incremental)
- ▶ Initialise it (randomly or with 0 is not a good choice)
- ▶ Apply update equation after each action
- ▶ Problem: it is (very) slow

Actor-critic: Naive design



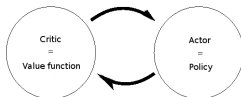
- ▶ Discrete states and actions, stochastic policy
- ▶ An update in the critic generates a local update in the actor
- ▶ Critic: compute δ and update $V(s)$ with $V_{k+1}(s) \leftarrow V_k(s) + \alpha_k \delta_k$
- ▶ Actor: $P_{k+1}^\pi(a|s) \leftarrow P_k^\pi(a|s) + \alpha_k \delta_k$
- ▶ NB: no need for a max over actions
- ▶ NB2: one must know how to “draw” an action from a probabilistic policy (not straightforward for continuous actions)



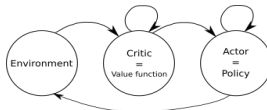
Williams, R. J. and Baird, L. (1990) A mathematical analysis of actor-critic architectures for learning optimal controls through incremental dynamic programming. In *Proceedings of the Sixth Yale Workshop on Adaptive and Learning Systems*, pages 96–101

Dynamic Programming and Actor-Critic

Policy Iteration



Actor-Critic



- ▶ In both PI and AC, the architecture contains a representation of the value function (the critic) and the policy (the actor)
- ▶ In PI, the MDP (T and r) is known
- ▶ PI alternates two stages:
 1. Policy evaluation: update $(V(s))$ or $(Q(s, a))$ given the current policy
 2. Policy improvement: follow the value gradient
- ▶ In AC, T and r are unknown and **not represented** (model-free)
- ▶ Information from the environment generates updates in the critic, then in the actor

From $Q(s, a)$ to Actor-Critic

state / action	a_0	a_1	a_2	a_3
e_0	0.66	0.88*	0.81	0.73
e_1	0.73	0.63	0.9*	0.43
e_2	0.73	0.9	0.95*	0.73
e_3	0.81	0.9	1.0*	0.81
e_4	0.81	1.0*	0.81	0.9
e_5	0.9	1.0*	0.0	0.9

state	chosen action
e_0	a_1
e_1	a_2
e_2	a_2
e_3	a_2
e_4	a_1
e_5	a_1

- ▶ Given a $Q - Table$, one must determine the max at each step
- ▶ This becomes expensive if there are numerous actions
- ▶ Store the best value for each state
- ▶ Update the max by just comparing the changed value and the max
- ▶ No more maximum over actions (only in one case)
- ▶ Storing the max is equivalent to storing the policy
- ▶ Update the policy as a function of value updates

Any question?



Send mail to: Olivier.Sigaud@isir.upmc.fr



Dayan, P. and Sejnowski, T. (1994).
TD(λ) converges with probability 1.
Machine Learning, 14(3):295–301.



Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. (2015).
Human-level control through deep reinforcement learning.
Nature, 518(7540):529–533.



Pardo, F., Tavakoli, A., Levdić, V., and Kormushev, P. (2018).
Time limits in reinforcement learning.
In *International Conference on Machine Learning*, pages 4045–4054. PMLR.



Puterman, M. L. (2014).
Markov decision processes: discrete stochastic dynamic programming.
John Wiley & Sons.



Roesch, M. R., Calu, D. J., and Schoenbaum, G. (2007).
Dopamine neurons encode the better option in rats deciding between differently delayed or sized rewards.
Nature neuroscience, 10(12):1615–24.



Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., et al. (2017).
Mastering chess and shogi by self-play with a general reinforcement learning algorithm.
arXiv preprint arXiv:1712.01815.



Singh, S. P., Jaakkola, T., Littman, M. L., and Szepesvári, C. (2000).
Convergence results for single-step on-policy reinforcement-learning algorithms.
Machine learning, 38(3):287–308.



Sutton, R. S. and Barto, A. G. (2018).
Reinforcement Learning: An Introduction (Second edition).
MIT Press.



Velentzas, G., Tzafestas, C., and Khamassi, M. (2017).

Bio-inspired meta-learning for active exploration during non-stationary multi-armed bandit tasks.
In *2017 Intelligent Systems Conference (IntelliSys)*, pages 661–669. IEEE.



Watkins, C. J. C. H. (1989).

Learning with Delayed Rewards.

PhD thesis, Psychology Department, University of Cambridge, England.



Watkins, C. J. C. H. and Dayan, P. (1992).

Q-learning.

Machine Learning, 8:279–292.



Williams, R. J. and Baird, L. (1990).

A mathematical analysis of actor-critic architectures for learning optimal controls through incremental dynamic programming.
In *Proceedings of the Sixth Yale Workshop on Adaptive and Learning Systems*, pages 96–101. Citeseer.